2023年
第1回プログラミングゼミ
UNIXコマンド,Awk

担当：D1　増川理恵

本日の目標

- ビッグデータの下処理に必要な知識と操作を身につける
  - **UNIXコマンド**を用いてデータファイルが操作できる
  - **Awk**でデータファイルの中身が処理できる

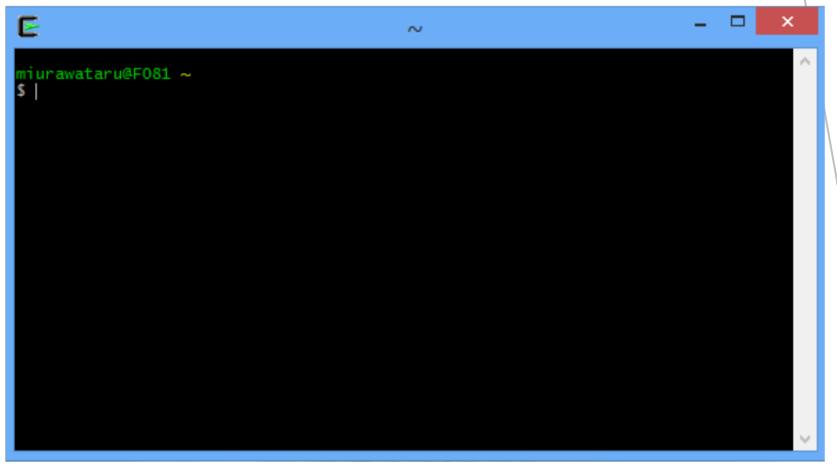
Linuxの開発環境

コマンドラインユーザーインターフェース（CUI）の操作

- Windows

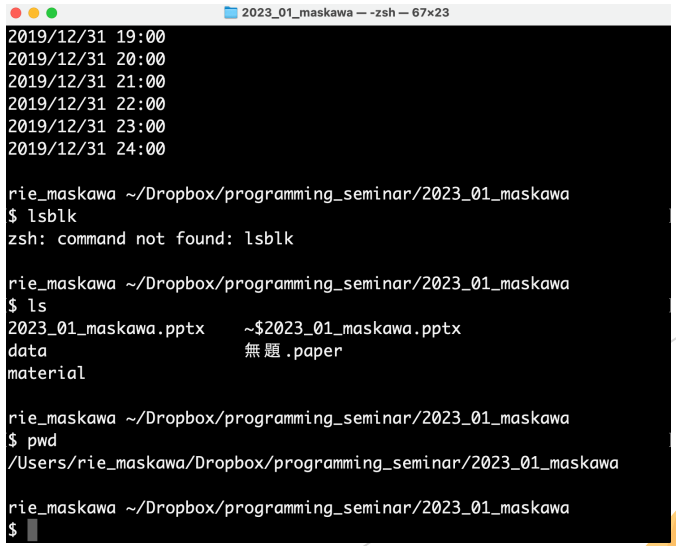
　WSL(Windows Subsystem Linux)を使用してLinux環境(Ubuntuなど)をインストールする

基本的にはWSL上での操作は、コマンドでの操作となる



- Mac

　Linuxの開発環境としてデフォルトでターミナルが入っているため設定不要



UNIXコマンド(≒Linuxコマンド)

ディレクトリ操作の基本コマンド

コマンド	意味
ls	一覧表示
cd	ディレクトリの移動
pwd	カレントディレクトリのパスを表示
mkdir	ディレクトリの作成

ls ファイルのリスト表示

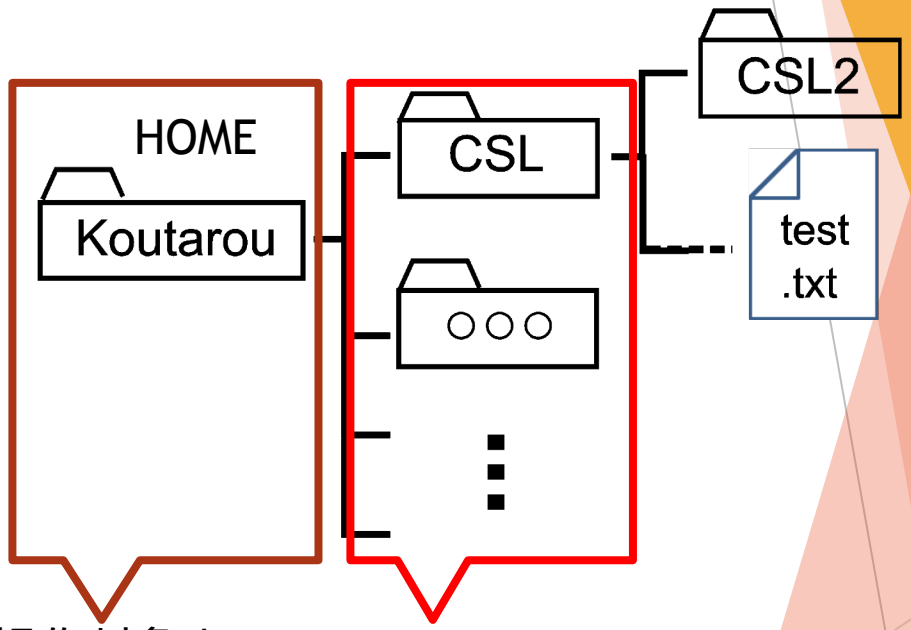
指定したディレクトリ内のファイルを一覧表示する
指定しなければカレントディレクトリの中身

ls [ディレクトリ名]

複数のディレクトリを指定することも可

```
$ ls data material
data:
2017                    2019
2018                    kanazawa2017-2019.csv

material:
2017            2017_2019       2018            2019            raw
```



現在操作対象となっているディレクトリ
(Current Directory)

コマンド「ls」で表示される一覧のファイル

cd ディレクトリの移動

cd [ディレクトリ名]

cd ..　ひとつ上のディレクトリへ移動
cd　　ホームディレクトリへ移動

cd
pwd

↑：以前のコマンド履歴
tab: 名前予測して補完
home: 一番左へ

ファイル表示の基本コマンド

コマンド	意味
head -n	ファイルの最初n行を表示
tail -n	ファイルの最後n行を表示
cat	ファイルの内容を全行表示
less	ファイルの内容をスクロールして表示

例　data/kanazawa2017-2019.csv の上から5行を表示

```
$ head -5 data/kanazawa2017-2019.csv
年月日時,気温(℃),降水量(mm),風速(m/s),風速(m/s),積雪(cm),降雪(cm),現地気圧(hPa)
2017/1/1 1:00,7.5,0.5,5.9,南南西,0,0,1020.4
2017/1/1 2:00,7.4,0,5.6,南南西,0,0,1020.6
2017/1/1 3:00,6.8,0.5,4.4,南南西,0,0,1020.8
2017/1/1 4:00,6.3,2.5,0.9,南南西,0,0,1020.7
```

プログラミングゼミ用のディレクトリを作成してデータを入れる

```
$ mkdir directory_name
```

データの中身をコマンドで確認してみる

課題のデータ詳細

気象庁公開のデータ
場所：　石川県金沢市
期間：　2017/1/1から2019/12/31
項目：気温(℃),降水量(mm),風速(m/s),風速(m/s),積雪(cm),降雪(cm),現地気圧(hPa)

kanazawa2017-2019.csv

	A	B	C	D	E	F	G	H
1	年月日時	気温(℃)	降水量(mr	風速(m/s)	風速(m/s)	積雪(cm)	降雪(cm)	現地気圧(hPa
2	2017/1/1 1:00	7.5	0.5	5.9	南南西	0	0	1020.4
3	2017/1/1 2:00	7.4	0	5.6	南南西	0	0	1020.6
4	2017/1/1 3:00	6.8	0.5	4.4	南南西	0	0	1020.8
5	2017/1/1 4:00	6.3	2.5	0.9	南南西	0	0	1020.7
6	2017/1/1 5:00	5.9	2	3.7	南	0	0	1021.2
7	2017/1/1 6:00	6.1	0	2.4	南南東	0	0	1021.7
8	2017/1/1 7:00	6.6	0	4.9	南南西	0	0	1022
9	2017/1/1 8:00	7.2	0	3.6	南西	0	0	1022.7
10	2017/1/1 9:00	7.7	0	1.6	南西	0	0	1022.8

そのまま開くとエクセル同様表形式のファイルだが、エディタで開くとカンマ区切りのデータファイルとなる。
（Comma Separated Value）

```
年月日時,気温(℃),降水量(mm),風速(m/s),風速(m/s),積雪(cm),降雪(cm),現地気圧(hPa)
2017/1/1 1:00,7.5,0.5,5.9,南南西,0,0,1020.4
2017/1/1 2:00,7.4,0,5.6,南南西,0,0,1020.6
2017/1/1 3:00,6.8,0.5,4.4,南南西,0,0,1020.8
2017/1/1 4:00,6.3,2.5,0.9,南南西,0,0,1020.7
2017/1/1 5:00,5.9,2,3.7,南,0,0,1021.2
2017/1/1 6:00,6.1,0,2.4,南南東,0,0,1021.7
2017/1/1 7:00,6.6,0,4.9,南南西,0,0,1022
2017/1/1 8:00,7.2,0,3.6,南西,0,0,1022.7
2017/1/1 9:00,7.7,0,1.6,南西,0,0,1022.8
2017/1/1 10:00,8.5,0,4.6,南南西,0,0,1022.9
2017/1/1 11:00,9.4,0,3.2,南西,0,0,1022.5
2017/1/1 12:00,8.2,0,4.2,南南西,0,0,1022
```

grep 指定文字列がある行を表示

grep オプション 検索文字列 ファイル

```
$ cat data/kanazawa2017-2019.csv | grep '南南西' | head -5
2017/1/1 1:00,7.5,0.5,5.9,南南西,0,0,1020.4
2017/1/1 2:00,7.4,0,5.6,南南西,0,0,1020.6
2017/1/1 3:00,6.8,0.5,4.4,南南西,0,0,1020.8
2017/1/1 4:00,6.3,2.5,0.9,南南西,0,0,1020.7
2017/1/1 7:00,6.6,0,4.9,南南西,0,0,1022
```

オプション	意味
-A num（行数）	一致した行の後の行（num で指定した行数）も表示
-B num（行数）	一致した行の前の行（num で指定した行数）も表示
-C num（行数）	一致した行の前後の行（num で指定した行数）も表示
-c	マッチした行の数を表示
-e pattern（検索文字列）	複数の検索文字列（パターン）を指定する場合に指定
-E	検索に拡張正規表現を使う
-f file（読み込むファイル）	検索する文字列（パターン）を指定したファイル（file）から読み込む
-i	大文字と小文字の区別をしない
-n	マッチした行のファイル内での行番号を表示
-r	サブディレクトリを含めて再帰的に検索
-v	マッチしなかった行を表示
-w	単語全体でパターンと一致するものを検索
-x	行全体とマッチする行のみを表示

grepなどで使える基本正規表現

メタ文字	意味
^	行の先頭。（例）^abc：abcから始まる行
$	行の終わり。（例）abc$：abcで終わる行。^$：空行
.	改行以外の任意の文字
*	直前文字の0回以上の繰り返し
[…]	[ ]で囲まれている文字のいずれか。（例） [abc123]：abc123のどれか
[^…]	[ ]で囲まれている文字以外。（例）[^abc123]：abc123以外
[n-n]	指定範囲（n-n）のいずれかの文字。（例）[a-c]：abcのどれか。 [1-3]：123のどれか
<	単語の始まり。（例）\<abc：abcで始まる単語。
\>	単語の終わり。（例）abc\>：abcで終わる単語。\<abc\>：abcという単語
\b	単語の区切り（単語境界）。（例）\bABC\b：ABCという単語
\s	任意の空白文字
\S	空白文字以外の文字
\w	任意の単語。[a-zA-Z0-9]と同じ
\W	単語文字以外の文字。[^a-zA-Z0-9]と同じ

複数のコマンド処理を組み合わせるパイプ

- パイプとは？　複数のコマンド処理を連続して使う便利なツール

```
Command1 file1 > file2
Command2 file2 > file3
Command3 file3 > file4
```

処理ごとに中間ファイルがどんどん生成されて行ってしまう。

上と同じ処理が1行で書ける！

そこで・・

```
Command1 file1 | Command2 | Command3 > file4
```

=cat file1 | command1 | command2 | command3

前のコマンドの結果を、後のコマンドの入力（ファイル読み込みに相当）として用いる

ワンライナー：パイプを最大限使用して、どんな複雑な処理も1行で書こうとする人種



ワンライナーへの道のり

UNIXコマンドでよく使うもの

コマンド	意味
ファイルやディレクトリの操作	
cp	コピー
mv	移動
rm	削除
chmod	アクセス権の変更
ファイルの中身に対する操作	
uniq	重複行を省略
sort	ファイルの中身を操作
wc	文字数や行数のカウント
grep	指定文字列の検索
sed	文字列の置換など

もっと自由にファイルの中身を操作したい

▶Awk

・UNIX上で開発されたプログラミング言語

・コンパイル不要

自分でスクリプトを記述することで、任意の機能を持つプログラムを作れる

例えば…

・列同士の四則演算

・指定された条件の行・列を抜き出す

・指定された条件に合致する行・列のデータを加工する

Awkの基本的な文法

awk ‘コマンド‘ ファイル

- **各行**に対して与えられた命令を実行する
- **$n** でn列目を表す

例：データの1列目を表示する

```
$ cat data/kanazawa2017-2019.csv | awk -F, '{print $1}'
年月日時
2017/1/1 1:00
2017/1/1 2:00
```

例：データの1列目×5と3列目を表示する

```
$ cat data/kanazawa2017-2019.csv | awk -F, '{print $2*5,$3}' |
0 降水量(mm)
37.5 0.5
37 0
```

Awkの基本的な文法

awk -F’区切り文字’ ‘BEGIN{ }{ }END{ }‘ ファイル

```
$ awk 'NR!=1{print $1}'
2017/1/1
2017/1/1
2017/1/1
```

→

```
$ awk -F, 'NR!=1{print $1}'
2017/1/1 1:00
2017/1/1 2:00
2017/1/1 3:00
```

・デフォルトの区切り文字は空白
・”は付けなくてもよい

Awkの基本的な文法

awk -F'区切り文字' 'BEGIN{ }{ }END{ }' ファイル

BEGIN{ファイルを読み込む前の処理}{メイン処理}END{ファイルを読み込んだ後の処理}

例　：　条件に合致した列の数を数える

```
$ awk -F, '{if($2>20)n++}END{print n/NR}'
```

- 変数は宣言なしで使える
- 組み込み変数
  NR:現在処理している行番号
  OFS:出力時の列の区切り文字

課題

(1)何年から何年までのデータがあるか？コマンドで確認

(2)年ごとにディレクトリを作成する

(3)年ごとにファイルを分割し、(2)のディレクトリに保存する

（ヒント）:grepやawkを使用

(4)年ごとの、1年の中で気温が20度を越えた時刻の割合

欠損値の確認

(5)年ごとの、気温の最小値・最大値・平均値・標準偏差

(6)全期間において、気温の前日差分の時系列を計算

(7)好きなデータにおいて月ごとの平均値を求める

(8)[発展課題]各日ごとに、気温の平均値、最小値、最大値、標準偏差を算出し、その時系列を作る。

正規表現

- grep 検索正規表現 ファイル名 で、正規表現にマッチする行全てを出力
- sed “s/正規表現/置換文字列/g”　でマッチした正規表現を置換

- ^　文頭　　$文末　　e.g.　2017　で始まる行は”^2017”

- . なんでもいい一文字　　* 直前の文字の繰り返し
- [0-9] 0-9の数字 ‘^[0-9]\{4,4\}‘ ４桁の数字
- “^[0-9]\{4,4\}/12”　　12月

シェルスクリプト

UNIXコマンドとAwkの組み合わせで**「ビッグデータの整理」**は何でもできる

◎より効率化、高速化する　：　シェルスクリプト

・一回一回、コマンドを打つのが面倒
・何をしたのか思い出せない

一連の作業を一つのファイルにまとめて書いて、まとめて実行できる

例　：　先ほどと同じ操作。

data_seiri.sh

```
nkf -w filename > filename2
sed “s/,/ /g” filename2 > filename3
rm filename2
```

sh data_seiri.shもしくは./data_seiri.sh と入力することで、
data_seiri.sh内の命令が順に実行される